

Análisis de Incidentes Viales de la Ciudad de México

2022 – 2024

Un tablero interactivo del análisis descriptivo al predictivo

Proyecto Final

Aplicaciones para Análisis de Datos

Julio Antonio Zavala

Junio 2026

1. Introducción

Los incidentes viales son uno de los problemas de seguridad y movilidad más relevantes de la Ciudad de México. Cada llamada de emergencia atendida por el C5 (Centro de Comando, Control, Cómputo, Comunicaciones y Contacto Ciudadano) queda registrada con su fecha, hora, ubicación, tipo de incidente y tiempo de atención. Analizar este registro permite entender *cuándo*, *dónde* y *qué* tipo de accidentes ocurren, información valiosa para la prevención y la asignación de recursos.

Este proyecto trabaja con **503,339 incidentes viales** registrados entre 2022 y 2024. La propuesta no se queda en describir los datos: los usa como fuente de conocimiento para construir un modelo capaz de estimar, dada una alcaldía y una hora, qué tipo de accidente es más probable. Todo el análisis se presenta en un **tablero interactivo** (dashboard) que permite a cualquier persona explorar los datos sin escribir una sola línea de código.

El documento describe la metodología seguida, el diseño orientado a objetos del sistema, los detalles de su implementación y los resultados obtenidos.

2. Objetivos

Objetivo general

Desarrollar un sistema en Python, orientado a objetos, que analice los incidentes viales de la CDMX (2022–2024) y, a partir del conocimiento descriptivo, prediga el tipo de accidente más probable según la zona y la hora, presentándolo en un tablero interactivo desplegable en la web.

Objetivos específicos

- Limpiar y preparar el conjunto de datos original, documentando cada decisión.
- Modelar el dominio con un diseño de clases (POO) que represente incidentes, ubicaciones, colonias, alcaldías y tipos de accidente.
- Realizar un análisis descriptivo que responda preguntas concretas mediante el tipo de gráfico adecuado a cada una.
- Construir, con el conocimiento descriptivo, un modelo predictivo del tipo de accidente, sin usar Random Forest, y evaluarlo de forma honesta.
- Integrar todo en un dashboard interactivo y desplegarlo en Streamlit Community Cloud para su acceso público.

3. Metodología, Diseño e Implementación

3.1 Metodología — ¿por qué este enfoque?

Se adoptó un proceso por fases inspirado en **CRISP-DM** (el estándar de facto en proyectos de datos): comprensión del problema, comprensión de los datos, preparación, modelado, evaluación y despliegue. Se eligió este marco porque obliga a un orden lógico: **no se puede modelar sobre datos**

sucios ni predecir sobre fenómenos que no se han entendido primero. Cada fase alimenta a la siguiente.

La decisión metodológica más importante del proyecto es ir **de lo descriptivo a lo predictivo**. En lugar de lanzar un algoritmo a “adivinar”, primero se describe el fenómeno para **generar conocimiento** —¿a qué horas hay más incidentes?, ¿qué zonas concentran cuáles tipos?— y solo entonces se predice, usando justamente las variables que el análisis descriptivo demostró relevantes (hora, alcaldía, mes y fin de semana). Así el modelo no es una caja negra: cada variable que recibe tiene una justificación previa. Este es además el requisito explícito del proyecto: que el predictivo **use** el conocimiento del descriptivo.

Preparación de los datos — por qué cada decisión de limpieza

El archivo original tenía 504,261 registros. La limpieza buscó conservar solo incidentes viales reales y bien formados, documentando cada criterio para que el proceso sea reproducible y defendible:

Decisión de limpieza	Por qué
Eliminar tipos no viales (Mi Taxi, Sismo, Mi Calle, Detención ciudadana)	No son accidentes de tránsito; contaminarían el análisis del fenómeno vial.
Eliminar registros de 2021 (73)	Quedan fuera del periodo de estudio (2022–2024); son ruido residual.
Eliminar duraciones negativas (3)	La fecha de cierre es anterior a la de creación: dato imposible, error de captura.
Marcar (no eliminar) atenciones > 24 h como outlier	Pueden ser reales; se etiquetan para excluirlas de promedios sin perder el registro.
Imputar colonia nula con “Sin colonia” (2.2%)	Conserva el incidente para el análisis temporal/tipo aunque falte la colonia exacta.
Derivar columnas (hora, mes, año, fin de semana, severidad, minutos de atención)	Convierten fechas crudas en variables directamente analizables y predictivas.

Agrupación en 7 tipos — por qué

La variable de subtipo traía 14 categorías, pero solo 6 concentran cerca del 99% de los casos; las otras 8 son una cola de eventos rarísimos. Se agruparon en **6 tipos principales + “Otros”**. El porqué es doble: **legibilidad** (un gráfico de pastel de 7 rebanadas se entiende; uno de 14 no) y **estabilidad del modelo** (clases con poquísimos casos generan predicciones poco fiables). Esta agrupación se encapsuló en una clase para reutilizarla en todo el sistema.

Preguntas y gráficos — por qué cada tipo de gráfico

El análisis descriptivo se planteó como seis preguntas, eligiendo en cada una el gráfico cuya naturaleza corresponde a la pregunta:

Pregunta	Gráfico	Por qué
¿Qué tipos predominan en una alcaldía?	Pastel	Muestra proporción de un todo (100%).

¿Dónde se concentran según la hora?	Mapa de calor	La densidad espacial se lee en un mapa, no en una tabla.
¿Cómo cambia el volumen durante el día?	Líneas / área	La hora es continua: la línea revela tendencia y picos.
¿Qué alcaldías tienen más y cuáles más graves?	Barras	Comparan magnitudes entre categorías discretas.
¿La atención se relaciona con el volumen?	Dispersión	Relaciona dos variables continuas; revela correlación.
¿Crece la demanda año con año?	Columnas agrupadas	Comparan la misma categoría (mes) entre series (años).

3.2 Diseño — ¿por qué este diseño de clases?

Se eligió la **programación orientada a objetos** porque el dominio se modela de forma natural como objetos que reflejan la realidad: un incidente ocurre en una ubicación y se reporta en un momento; una colonia agrupa incidentes; una alcaldía agrupa colonias. La POO permite **encapsular** los datos junto con el comportamiento (cada clase sabe calcular sus propias métricas), hace el código **reutilizable** y deja que el descriptivo y el predictivo compartan la misma representación del conocimiento.

El sistema se organiza en tres capas. El diagrama UML completo está en [docs/UML_clases.svg](#).

Clase	Capa	Responsabilidad (por qué existe)
UbicacionGeografica	dominio	Aísla lat/long, alcaldía y colonia; calcula distancias (Haversine).
ReporteC4	dominio	Aísla el “cuándo y cómo” del reporte: hora, franja, tiempo de respuesta.
Incidente	dominio	Objeto atómico; compone una ubicación y un reporte.
Colonia	dominio	Agrega incidentes y expone métricas locales.
Alcaldia	dominio	Agrega colonias; métricas de demarcación.
TipoAccidente	conocimiento	Encapsula uno de los 7 tipos y su distribución (hora, zona, gravedad).
CatalogoTipos	conocimiento	Agrega los 7 tipos; se construye vectorizado desde los datos.
AnalisisZona	predictivo	Opera sobre una alcaldía (análisis y agrupamientos).
PredictorTipoAccidente	predictivo	El modelo: Naive Bayes + baseline de frecuencias.

Composición vs. agregación — por qué cada relación es la que es

Distinguir estas dos relaciones fue una decisión de diseño consciente, no estética:

- **Composición** (el todo posee la parte y controla su ciclo de vida): *Incidente* compone su *ReporteC4* y su *UbicacionGeografica*, porque un reporte o una ubicación **no tienen sentido sin su incidente**: nacen y mueren con él.

- **Agregación** (el todo agrupa partes que existen por sí mismas): *Colonia* agrega *Incidentes*, *Alcaldía* agrega *Colonias* y *CatalogoTipos* agrega *TipoAccidente*. Los incidentes existen independientemente de la colonia que los agrupe para un análisis.

La clase TipoAccidente — por qué se añadió

Es la clase nueva del proyecto (sugerida por la asesoría) y la pieza clave del enfoque: actúa como **punto entre el descriptivo y el predictivo**. Concentra todo el conocimiento estadístico de un tipo de accidente —su porcentaje, su tasa de gravedad, su distribución por hora y por alcaldía— de modo que el descriptivo la usa para dibujar el pastel y el predictivo usa su distribución horaria como evidencia. El conocimiento se calcula una sola vez y se reutiliza, en lugar de repetir la lógica en cada módulo.

3.3 Implementación — ¿por qué se implementó así?

Tecnologías y por qué

- **pandas / NumPy**: manejo vectorizado de medio millón de filas.
- **scikit-learn**: el modelo Naive Bayes y las métricas de evaluación.
- **Streamlit**: convierte un script de Python en una aplicación web interactiva sin programar frontend, y se despliega gratis en la nube.
- **Plotly**: gráficos interactivos (hover, zoom) y un mapa que no requiere token de pago para mostrarse.
- **PyArrow / Parquet**: formato de datos comprimido para el despliegue.

Rendimiento: POO sobre agregados, no sobre 500 mil objetos

Construir 503,339 objetos *Incidente* en memoria haría el tablero lento. Por eso las clases de conocimiento (*CatalogoTipos*) se construyen de forma **vectorizada** directamente desde el DataFrame con un método de fábrica ([desde_dataframe](#)): se conserva el diseño orientado a objetos, pero el cálculo es prácticamente instantáneo. La POO aporta la *estructura* y el *significado*; pandas aporta la *velocidad*.

Dos formatos de datos: CSV canónico y Parquet para la nube

El CSV limpio (la fuente canónica del proyecto, ~128 MB) es el resultado auditable de la limpieza y el que alimenta el pipeline de clases. Sin embargo, GitHub rechaza archivos de más de 100 MB y el tablero debe cargar rápido en la nube. Por eso se genera además un **Parquet** (~9 MB) con tipos optimizados (categorías, enteros de 8 bits, flotantes de 32): es un espejo ligero del CSV. El dashboard lee el Parquet; el análisis local puede usar cualquiera de los dos.

El modelo predictivo — por qué Naive Bayes y un baseline

El requisito era predecir el tipo de accidente **sin Random Forest** y aprovechando el conocimiento descriptivo. La solución combina dos vías:

- **Baseline (frecuencias condicionadas)**: la probabilidad $P(\text{tipo} \mid \text{alcaldía, hora})$ leída directamente de los datos. Es, literalmente, el conocimiento descriptivo en bruto.

- **Modelo (Naive Bayes categórico):** la versión *suavizada* (con suavizado de Laplace) y *generalizadora* de esas mismas frecuencias. Se eligió Naive Bayes porque es la contraparte probabilística **directa** de un conteo de frecuencias —no una caja negra— y entrega una probabilidad para cada tipo, que es justo lo que el dashboard muestra.

Para las combinaciones (alcaldía, hora) con pocos casos se aplica un **respaldo jerárquico**: si la celda exacta es escasa, se mezcla con el comportamiento general de la alcaldía y, en último caso, con el global. Esto evita predicciones erráticas por falta de datos.

El tablero: interactividad y despliegue

La aplicación se divide en páginas (Inicio, Dataset, Descriptivo, Predictivo) y usa el sistema de **caché** de Streamlit para cargar los datos y entrenar el modelo una sola vez. El resultado se publica en **Streamlit Community Cloud**, que instala las dependencias automáticamente y entrega una URL pública: el usuario solo abre el enlace, sin instalar nada.

4. Resultados

El análisis descriptivo arrojó hallazgos claros, que el modelo predictivo después cuantifica:

- **Distribución de tipos:** los choques dominan el panorama —“sin lesionados” (45.9%) y “con lesionados” (29.1%)—, seguidos de Atropellado (10.4%) y Motociclista (10.0%).
- **Gravedad:** el 10.6% de los incidentes son graves (lesionados o fallecidos). Un dato estructural revelador: prácticamente **todos los atropellamientos se clasifican como graves**.
- **Hora pico:** las 19:00 h concentran el mayor número de incidentes; el volumen sube de forma sostenida durante el día y cae de madrugada.
- **Geografía:** *Iztapalapa* es la alcaldía con más incidentes (82,068), mientras que *Cuauhtémoc* tiene la mayor tasa de gravedad (15.3%). Volumen y gravedad no son lo mismo.

El modelo predictivo confirma y cuantifica ese conocimiento. Por ejemplo, en **Cuauhtémoc a las 19:00 h** la probabilidad de *Atropellado* sube a ~16% frente al ~10% global: la zona y la hora sí cambian el perfil de riesgo.

Evaluación honesta del modelo

El modelo alcanza una exactitud (accuracy) de 0.46 y un F1-macro bajo. Lejos de ser un fracaso, esto es un **hallazgo**: como “Choque sin lesionados” es casi la mitad de todos los casos, predecir *el tipo más probable* casi siempre devuelve esa clase mayoritaria. Por eso el valor del predictor no está en acertar una sola etiqueta, sino en la **distribución de probabilidades** completa, que sí refleja cómo cambian los riesgos por zona y hora. Reportarlo así, en lugar de inflar una métrica, es parte del rigor del proyecto.

5. Conclusión

El proyecto cumple su objetivo: convierte medio millón de registros crudos en conocimiento accionable y lo pone al alcance de cualquiera mediante un tablero interactivo. La decisión de ir **de lo descriptivo a lo predictivo** demostró su valor: cada variable del modelo está justificada por un hallazgo previo, y el predictor no hace más que formalizar lo que los gráficos ya sugerían.

El diseño orientado a objetos —con la distinción cuidadosa entre composición y agregación, y la clase *TipoAccidente* como puente del conocimiento— mantuvo el código ordenado y reutilizable. En la implementación, separar el CSV canónico del Parquet ligero, y construir las clases de forma vectorizada, permitió un tablero rápido y desplegable en la nube sin sacrificar el rigor del modelado.

Como trabajo futuro, el modelo podría enriquecerse con variables como el clima o la presencia de eventos masivos, y el análisis de gravedad podría abordarse como un problema propio. Aun así, el sistema actual entrega una base sólida, interpretable y honesta para entender los incidentes viales de la CDMX.